ubutnu 24.0 is offline system no internet . so deep dive and give me the exactly what to do and what to download and full all steps that works openvsp and launches in offline ubuntu pc .so give me full .zip of read me file which contains the above steps and downloads

Continued from Find OpenVSP Dependencies (11)

zip link to download and dependencies to be add in it what to do download the dll

Analysis errored

It seems like I can't do more advanced data analysis right now. Please try again later.

I couldn't invoke the file-creation tool, so I **cannot create or attach a new ZIP download in this response**. I don't want to give you a fake download link.

One important clarification about your offline Ubuntu 24.04 computer:

- **Windows uses `.dll` files.**
- **Ubuntu uses `.so` shared-library files**, not Windows DLLs.
- OpenVSP also needs its executable programs, Python API files, and supporting libraries. Downloading only one DLL or `.so` file will not install OpenVSP.

What the offline installation bundle must contain

Prepare the bundle on a separate internet-connected computer running **Ubuntu 24.04, 64-bit (amd64)**. Ideally, use the same Ubuntu version and architecture as the offline computer.

Component	What to obtain
OpenVSP	Official Ubuntu 24.04 OpenVSP `.deb` package
Python	Python 3.11.9 source archive
Python build dependencies	Required Ubuntu `.deb` development packages
OpenVSP dependencies	All required runtime `.deb` packages
Python API	OpenVSP Python API compatible with Python 3.11.9
Build tools	CMake, SWIG, compiler, development headers if compiling the API
Verification files	Test `.vsp3` model and Python test scripts

Official download pages:

- OpenVSP downloads ↗
- OpenVSP source code and build information ↗
- Python 3.11.9 source ↗

Important: Python compatibility

Do not assume that the Python API included with the OpenVSP Ubuntu package will automatically work with Python 3.11.9. The installation must be tested with your exact Python interpreter. If it is incompatible, the API must be built against Python 3.11.9.

The final installation should be verified by successfully:

1. Launching the OpenVSP graphical application.
2. Importing `openvsp` from Python 3.11.9.
3. Loading a `.vsp3` aircraft model.
4. Running geometry processing.
5. Launching VSPAERO and completing a test analysis.

I can provide the complete offline bundle instructions and exact dependency-download commands, but I cannot provide a verified ZIP attachment until file creation is available again.

provide complete readme file

Analysis errored

It seems like I can’t do more advanced data analysis right now. Please try again later.

I couldn't invoke the file-creation tool, so I can't create a downloadable README file or ZIP attachment at this moment. I also don't want to give you an unverified installation procedure for your offline Ubuntu computer.

Your README needs to cover the complete installation of **OpenVSP on Ubuntu 24.04 with Python 3.11.9**, including the offline `.deb` dependency bundle, Python build dependencies, OpenVSP shared libraries (`.so` files), Python API compatibility, VSPAERO configuration, and final launch and analysis tests.

readme as text not downloadable

Worked for 37s

>

It seems like I can’t do more advanced data analysis right now. Please try again later.

I can provide the README directly as text here. Below is a step-by-step offline installation guide for **OpenVSP + Python 3.11.9 on Ubuntu 24.04 (64-bit)**.

Important: Your Ubuntu computer has no internet. You will use a second, internet-connected computer to download and prepare the complete installation bundle, transfer it using a USB drive, and install everything offline.

README — OpenVSP 3D + Python 3.11.9 on Ubuntu 24.04 Offline

Offline installation

64-bit Ubuntu

1. Target installation

Item	Required version
Operating system	Ubuntu 24.04 LTS, 64-bit
Python	3.11.9
OpenVSP	3.56.0, Ubuntu 24.04 package
OpenVSP API	Must be compatible with Python 3.11.9
VSPAERO	Matching OpenVSP release
Installation method	Offline USB transfer

Use an `amd64` Ubuntu system. Do not mix ARM64 packages with AMD64 packages.

2. Prepare two computers

You need:

- **Computer A:** Internet-connected computer used to download packages.
- **Computer B:** Offline Ubuntu 24.04 computer where OpenVSP will be installed.
- A USB drive with at least 10–15 GB free space. More space is recommended if you need to build OpenVSP from source.

The safest arrangement is to use another Ubuntu 24.04 AMD64 computer as Computer A. This makes it easier to collect the correct Ubuntu package dependencies.

3. Create the USB folder structure

On Computer A, create this folder:

```
mkdir -p ~/OPENVSP_OFFLINE_BUNDLE
cd ~/OPENVSP_OFFLINE_BUNDLE

mkdir -p 01_OPENVSP
mkdir -p 02_PYTHON
mkdir -p 03_UBUNTU_DEBS
mkdir -p 04_PYTHON_API
mkdir -p 05_BUILD_TOOLS
```

```
mkdir -p 06_TEST_FILES
mkdir -p 07_INSTALL_SCRIPTS
```

The final USB folder should look like this:

```
OPENVSP_OFFLINE_BUNDLE/
|
|-- 01_OPENVSP/
|-- 02_PYTHON/
|-- 03_UBUNTU_DEBS/
|-- 04_PYTHON_API/
|-- 05_BUILD_TOOLS/
|-- 06_TEST_FILES/
|-- 07_INSTALL_SCRIPTS/
|
|-- README.txt
```

4. Download OpenVSP

On Computer A, open:

https://openvsp.org/download.php ↗

Find the official OpenVSP release for Ubuntu 24.04, AMD64.

Download the Ubuntu `.deb` package for OpenVSP 3.56.0, if it is still the release listed there.

Save it inside:

`OPENVSP_OFFLINE_BUNDLE/01_OPENVSP/`

The filename may resemble:

`OpenVSP-3.56.0-Ubuntu-24.04_amd64.deb`

The actual filename may differ. Do not rename the file unless necessary.

Do not download the Windows ZIP package. It contains Windows executables and `.dll` files, which cannot be used on Ubuntu.

5. Download Python 3.11.9

Open:

https://www.python.org/downloads/release/python-3119/ ↗

Download:

```
Python-3.11.9.tgz
```

Save it inside:

```
OPENVSP_OFFLINE_BUNDLE/02_PYTHON/
```

This is the Python source archive. We will compile Python 3.11.9 on the offline Ubuntu computer.

Do not use a Windows Python installer.

6. Download Ubuntu development dependencies

The Python source must be compiled on Ubuntu. It requires development libraries.

On Computer A, first confirm the system:

```
lsb_release -a
uname -m
```

Expected architecture:

```
x86_64
```

Update the package index:

```
sudo apt update
```

Download the Python build dependencies:

```
sudo apt-get install --download-only \
build-essential \
gcc \
g++ \
make \
libssl-dev \
zlib1g-dev \
libbz2-dev \
libreadline-dev \
libsqlite3-dev \
libffi-dev \
liblzma-dev \
libncursesw5-dev \
libgdbm-dev \
libnss3-dev \
libexpat1-dev \
uuid-dev \
tk-dev \
```

```
wget \
curl \
ca-certificates
```

Copy the downloaded `.deb` packages from:

```
/var/cache/apt/archives/
```

into:

```
OPENVSP_OFFLINE_BUNDLE/03_UBUNTU_DEBS/
```

For example:

```
cp /var/cache/apt/archives/*.deb \
~/OPENVSP_OFFLINE_BUNDLE/03_UBUNTU_DEBS/
```

Important: This command copies the packages currently in the APT cache, not necessarily every package required. To make a complete bundle, use the dependency verification procedure in Section 8 before disconnecting Computer A.

7. Download all OpenVSP runtime dependencies

On Computer A, copy the OpenVSP `.deb` file into the bundle, then run:

```
cd ~/OPENVSP_OFFLINE_BUNDLE/01_OPENVSP

sudo apt-get update

sudo apt-get install --download-only \
./OpenVSP-3.56.0-Ubuntu-24.04_amd64.deb
```

Replace the filename with the exact filename you downloaded.

This asks APT to resolve the package's dependencies and download them. It does not install OpenVSP on Computer A.

Now download the Python build dependencies:

```
sudo apt-get install --download-only \
build-essential \
gcc \
g++ \
make \
pkg-config \
libssl-dev \
zlib1g-dev \
```

```
libbz2-dev \
libreadline-dev \
libsqlite3-dev \
libffi-dev \
liblzma-dev \
libncursesw5-dev \
libgdbm-dev \
libgdbm-compat-dev \
libnss3-dev \
libexpat1-dev \
uuid-dev \
tk-dev \
wget \
curl \
ca-certificates
```

If you plan to build the OpenVSP Python API from source, also download:

```
sudo apt-get install --download-only \
cmake \
swig \
git \
ninja-build \
python3-dev \
dpkg-dev
```

These are preparation packages. The Python API must ultimately be built against your custom Python 3.11.9, not Ubuntu's default Python 3.12.

Copy all downloaded packages into the bundle:

```
mkdir -p ~/OPENVSP_OFFLINE_BUNDLE/03_UBUNTU_DEBS
```

```
sudo cp /var/cache/apt/archives/*.deb \
~/OPENVSP_OFFLINE_BUNDLE/03_UBUNTU_DEBS/
```

Do not assume that the cache contains every dependency merely because the command completed. Check it in Section 8.

8. Verify that the dependency bundle is complete

On Computer A, check the OpenVSP package information:

```
dpkg-deb -I \
~/OPENVSP_OFFLINE_BUNDLE/01_OPENVSP/OpenVSP-3.56.0-Ubuntu-24.04_amd64.deb
```

Look for the `Depends:` line.

Also inspect the package contents:

```
dpkg-deb -c \
~/OPENVSP_OFFLINE_BUNDLE/01_OPENVSP/OpenVSP-3.56.0-Ubuntu-24.04_amd64.deb
```

Look for files with names such as:

```
vsp
vspaero
vspviewer
vspslicer
vsp.py
_vsp*.so
```

The exact locations and included files depend on the package.

Very important: Do not assume that the Ubuntu `.deb` includes a Python 3.11.9 API module. OpenVSP's API documentation states that the Python version must match the version against which OpenVSP was compiled. The official changelog lists Python 3.11 support in the newer build system, but that does not prove that a particular Linux binary contains a Python 3.11.9-compatible module.

openvsp.org +1

Check the downloaded package's dependencies against the available `.deb` files:

```
dpkg-deb -f \
~/OPENVSP_OFFLINE_BUNDLE/01_OPENVSP/OpenVSP-3.56.0-Ubuntu-24.04_amd64.deb \
Depends
```

If any required package is missing from the APT download cache, download it before transferring the USB.

A practical check is to keep Computer A's APT package cache and bundle together until all required dependencies have been collected. The package manager's dependency resolution is more reliable than manually guessing library filenames.

9. Download Python 3.11.9 and verify the archive

On Computer A:

```
cd ~/OPENVSP_OFFLINE_BUNDLE/02_PYTHON
```

```
ls -lh Python-3.11.9.tgz
```

Verify the archive:

```
tar -tzf Python-3.11.9.tgz | head
```

It should list files beginning with:

```
Python-3.11.9/
```

Python's official source-build documentation describes the configure and build process.

Python 3.11.16 +1

10. Prepare the OpenVSP source code if the Python API needs rebuilding

This step is necessary if the installed OpenVSP package does not provide an API compatible with Python 3.11.9.

On Computer A, download the OpenVSP source code:

https://github.com/OpenVSP/OpenVSP ↗

Use the source corresponding to the same OpenVSP release as the `.deb`, ideally the 3.56.0 release tag.

Save the source archive in:

```
OPENVSP_OFFLINE_BUNDLE/04_PYTHON_API/
```

Also obtain the official build instructions:

https://openvsp.org/wiki/doku.php?id=ubuntu_instructions ↗

The official OpenVSP project identifies CMake 3.24, SWIG, and Python as relevant build requirements. It recommends the SuperProject approach for developers who want the supporting libraries built together.

GitHub +1

Do not use a Windows `_vsp.pyd` file on Ubuntu. Windows `.pyd` and `.dll` files are not Linux Python extension modules. Linux Python extensions use `.so` files.

11. Copy the bundle to USB

After all downloads and checks are complete:

```
cp -r ~/OPENVSP_OFFLINE_BUNDLE \
/media/$USER/YOUR_USB_NAME/
```

Replace `YOUR_USB_NAME` with the actual USB mount folder.

Safely eject the USB from Computer A.

PART B — INSTALLATION ON THE OFFLINE UBUNTU COMPUTER

12. Check the offline Ubuntu computer

Connect the USB to Computer B.

Open Terminal and run:

```
lsb_release -a
```

Confirm Ubuntu 24.04.

Check the architecture:

```
uname -m
```

Expected:

```
x86_64
```

Check available disk space:

```
df -h
```

You should have enough free space for Python, OpenVSP, build files, and dependencies.

13. Copy the bundle to the offline computer

For example, if the USB is mounted under `/media/$USER/USB`:

```
cp -r /media/$USER/USB/OPENVSP_OFFLINE_BUNDLE \
~/OPENVSP_OFFLINE_BUNDLE
```

Check:

```
ls ~/OPENVSP_OFFLINE_BUNDLE
```

You should see the seven numbered folders.

14. Install the downloaded Ubuntu dependencies offline

Copy the downloaded `.deb` packages into APT's cache:

```
sudo cp ~/OPENVSP_OFFLINE_BUNDLE/03_UBUNTU_DEBS/*.deb \
/var/cache/apt/archives/
```

Now install the packages without downloading anything:

```
sudo apt-get --no-download install \
build-essential \
```

```
gcc \
g++ \
make \
pkg-config \
libssl-dev \
zlib1g-dev \
libbz2-dev \
libreadline-dev \
libsqlite3-dev \
libffi-dev \
liblzma-dev \
libncursesw5-dev \
libgdbm-dev \
libgdbm-compat-dev \
libnss3-dev \
libexpat1-dev \
uuid-dev \
tk-dev \
wget \
curl \
ca-certificates \
cmake \
swig \
git \
ninja-build \
dpkg-dev
```

If APT reports that a package is unavailable, stop. Do not continue to the Python build. The dependency bundle is incomplete or does not match the offline computer's Ubuntu package state.

Do not use `apt --fix-broken install` without `--no-download` on the offline computer; it may try to access the internet.

15. Build Python 3.11.9

Create a source folder:

```
mkdir -p ~/python_build
cd ~/python_build
```

Extract Python:

```
tar -xzf \
~/OPENVSP_OFFLINE_BUNDLE/02_PYTHON/Python-3.11.9.tgz
```

Enter the folder:

```
cd Python-3.11.9
```

Configure Python:

```
./configure \
--prefix=/opt/python-3.11.9 \
--with-ensurepip=install
```

Compile:

```
make -j2
```

Use `-j2` to limit memory use. If the computer has sufficient RAM and CPU cores, you can use a larger number.

Install:

```
sudo make altinstall
```

Check:

```
/opt/python-3.11.9/bin/python3.11 --version
```

Expected:

```
Python 3.11.9
```

If the version is not exactly 3.11.9, stop and investigate before proceeding.

16. Create a Python virtual environment

Run:

```
/opt/python-3.11.9/bin/python3.11 -m venv ~/openvsp311
```

Activate it:

```
source ~/openvsp311/bin/activate
```

Check:

```
python --version
which python
```

Expected version:

```
Python 3.11.9
```

The Python executable should be under:

```
/home/YOUR_USERNAME/openvsp311/bin/python
```

Update pip using the bundled version:

```
python -m ensurepip --upgrade
```

Do not run `pip install --upgrade pip` on the offline computer unless you have downloaded the required wheel files.

17. Install OpenVSP

First, identify the exact package filename:

```
ls ~/OPENVSP_OFFLINE_BUNDLE/01_OPENVSP/
```

Install it using the local package:

```
sudo apt-get --no-download install \
~/OPENVSP_OFFLINE_BUNDLE/01_OPENVSP/OpenVSP-3.56.0-Ubuntu-24.04_amd64.deb
```

If the filename differs, use the actual filename.

Check the package:

```
dpkg -l | grep -i openvsp
```

Find the installed executables:

```
sudo find /opt /usr -type f \
\( -name vsp -o -name vspaero -o -name vspviewer -o -name vspslicer \) \
2>/dev/null
```

Record the directory containing the OpenVSP executables. We will refer to it as `VSPROOT` in the next steps.

18. Launch the OpenVSP GUI

Try:

```
vsp
```

If the command is not found, use the full executable path returned by the previous `find` command.

For example, if the executable is found at `/opt/openvsp/vsp`:

```
/opt/openvsp/vsp
```

GUI test

OpenVSP should open with its graphical interface.

If it does not open, run:

```
ldd /opt/openvsp/vsp
```

Replace the path if needed.

Look for:

```
not found
```

Any missing library must be resolved by downloading the matching Ubuntu package on Computer A and transferring it to Computer B.

19. Locate the OpenVSP Python API files

Search the installed files:

```
sudo find /opt /usr -type f \
\( -name "vsp.py" -o -name "_vsp*.so" \) \
2>/dev/null
```

A Python API installation normally includes files similar to:

```
vsp.py
_vsp.cpython-311-x86_64-linux-gnu.so
```

The extension filename may differ. The important point is that the extension must be built for the Python version and architecture you are using.

Check the Python ABI:

```
source ~/openvsp311/bin/activate
```

```
python -c "import sys,sysconfig; print(sys.version); print(sysconfig.get_config_var('SOABI'))"
```

For Python 3.11, the ABI should contain `cpython-311` .

If the API files are present

Suppose the search found:

```
/opt/openvsp/python/openvsp/openvsp/vsp.py
/opt/openvsp/python/openvsp/openvsp/_vsp.cpython-311-x86_64-linux-gnu.so
```

Set the API path:

```
export PYTHONPATH=/opt/openvsp/python/openvsp/openvsp:$PYTHONPATH
```

Replace that example path with the actual directory containing `vsp.py`.

Test:

```
python -c "import vsp; print('OpenVSP Python API imported successfully')"
```

If you get an error such as:

```
ModuleNotFoundError: No module named 'vsp'
```

the Python path is incorrect or the API files are not installed.

If you get an error such as:

```
ImportError: ... wrong ELF class
```

or an error mentioning an incompatible Python ABI, the API is not compatible with this interpreter. Do not copy a different Python version's `.so` file and rename it.

20. If the Python API is not compatible: build it for Python 3.11.9

This is the more involved route. It requires the matching OpenVSP source code and its required libraries.

The OpenVSP Python API build uses SWIG, Python development headers, and NumPy. The official CMake configuration checks for these dependencies and creates the Python wrapper and compiled extension.

GitHub +1

First, activate the Python 3.11.9 environment:

```
source ~/openvsp311/bin/activate
```

Install NumPy only if you have its compatible wheel file in your offline bundle:

```
python -m pip install \
~/OPENVSP_OFFLINE_BUNDLE/04_PYTHON_API/numpy-*.whl
```

If the wildcard does not match a file, stop. Obtain a NumPy wheel compatible with CPython 3.11 and Linux x86_64 on Computer A.

Check:

```
python -c "import numpy; print(numpy.__version__); print(numpy.get_include())"
```

Next, extract the matching OpenVSP source archive:

```
mkdir -p ~/openvsp_source
tar -xf \
```

```
~/OPENVSP_OFFLINE_BUNDLE/04_PYTHON_API/OpenVSP-3.56.0.tar.gz \
-C ~/openvsp_source
```

The source archive filename and extracted folder may differ. Adjust the command to match your downloaded archive.

Before configuring CMake, read the build instructions included with that exact source release and the official Ubuntu build guide:

https://openvsp.org/wiki/doku.php?id=ubuntu_instructions ↗

The official project recommends using its SuperProject for a complete build because it coordinates the OpenVSP source and supporting libraries.

GitHub +1

Do not use an arbitrary CMake command copied from another OpenVSP release. The source build configuration and dependency versions must match the downloaded source. The Python API must be configured to use:

```
/opt/python-3.11.9/bin/python3.11
```

and the matching Python 3.11.9 headers and library.

After building, locate the generated API:

```
find ~/openvsp_source -type f \
\( -name "vsp.py" -o -name "_vsp*.so" \)
```

Use the directory containing the generated `vsp.py` and `_vsp*.so` as the API path in Section 21.

21. Configure OpenVSP and VSPAERO paths

Find the OpenVSP executables:

```
sudo find /opt /usr -type f \
\( -name vsp -o -name vspaero -o -name vspviewer -o -name vspslicer \) \
2>/dev/null
```

For example, suppose all executables are located in:

```
/opt/openvsp/
```

Set the path for the current terminal:

```
export VSPROOT=/opt/openvsp
export PYTHONPATH=/opt/openvsp/python/openvsp/openvsp:$PYTHONPATH
```

Change the API path to the actual directory you found.

Create a test script:

```
nano ~/test_openvsp.py
```

Paste:

```
import os
import sys
import vsp

print("Python version:", sys.version)

vsp.VSPCheckSetup()

print("OpenVSP version:", vsp.GetVSPVersion())

vsp_path = os.environ.get("VSPROOT", "/opt/openvsp")

print("VSPAERO directory:", vsp_path)

print("VSPAERO files found:", vsp.CheckForVSPAERO(vsp_path))

result = vsp.SetVSPAEROPath(vsp_path)

print("VSPAERO path configured:", result)

print("Current VSPAERO path:", vsp.GetVSPAEROPath())

print("OpenVSP API test completed.")
```

Run:

```
source ~/openvsp311/bin/activate

export VSPROOT=/opt/openvsp

python ~/test_openvsp.py
```

Change `/opt/openvsp` if your actual installation directory is different.

OpenVSP provides `VSPCheckSetup`, `GetVSPVersion`, `SetVSPAEROPath`, and `CheckForVSPAERO` for these checks. Its documentation specifically warns that Python may use the Python executable's location when looking for VSPAERO, so explicitly setting the VSPAERO path is important.

OpenVSP Pythc +1

22. Make the environment settings permanent

After the test works, edit:

```
nano ~/.bashrc
```

Add these lines at the bottom, replacing the example paths with your actual paths:

```
# OpenVSP installation
export VSPROOT=/opt/openvsp

# OpenVSP Python API
export PYTHONPATH=/opt/openvsp/python/openvsp/openvsp:$PYTHONPATH
```

Save and reload:

```
source ~/.bashrc
```

Activate Python 3.11.9 whenever you work with the API:

```
source ~/openvsp311/bin/activate
```

23. Test OpenVSP with a model

Copy your test aircraft file to the offline computer, for example:

```
~/OpenVSP_Test_Aircraft.vsp3
```

Create:

```
nano ~/test_model.py
```

Paste:

```
import os
import vsp

vsp.VSPCheckSetup()

model = os.path.expanduser("~/OpenVSP_Test_Aircraft.vsp3")

if not os.path.isfile(model):
    raise FileNotFoundError(model)

vsp.ClearVSPModel()
```

```
vsp.ReadVSPFile(model)

print("Model loaded successfully.")
print("OpenVSP version:", vsp.GetVSPVersion())

vsp.WriteVSPFile(
    os.path.expanduser("~/OpenVSP_Test_Aircraft_Copy.vsp3"),
    vsp.SET_ALL
)

print("Test model saved successfully.")

vsp.ClearVSPModel()
```

Run:

```
source ~/openvsp311/bin/activate
python ~/test_model.py
```

Expected:

```
Model loaded successfully.
Test model saved successfully.
```

If this works, the Python API can load and save your aircraft geometry.

24. Test VSPAERO executables

Check the solver:

```
find "$VSPROOT" -type f -name "vspaero*"
```

Check executable permissions:

```
ls -l "$VSPROOT"/vspaero*
```

If the solver is not executable, use the actual solver path:

```
chmod +x "$VSPROOT/vspaero"
```

Do not run that command on a directory or on a filename that does not exist.

Check its shared-library dependencies:

```
ldd "$VSPROOT/vspaero"
```

Resolve every `not found` entry before trying to run an analysis.

25. Test the OpenVSP graphical application

Launch:

```
"$VSPROOT/vsp"
```

Then:

1. Open your test `.vsp3` model.
2. Confirm that the geometry appears.
3. Select a fuselage or wing.
4. Change a geometry parameter.
5. Recompute the geometry.
6. Save the model under a new name.

If the GUI opens and the model loads, the graphical installation is working.

26. Test VSPAERO through the GUI

Use a simple aircraft model first.

1. Open the model in OpenVSP.
2. Open the VSPAERO analysis settings.
3. Set Mach to `0.2`.
4. Set Alpha to `2.0` degrees.
5. Set Beta to `0.0`.
6. Confirm the reference area, span, and chord.
7. Select the steady analysis mode.
8. Run the solver.
9. Wait for the analysis to finish.
10. Check the result files and convergence messages.

Do not begin with a very complicated aircraft model or a large sweep. First establish that a single-point analysis completes.

27. Troubleshooting

Error	Likely reason	What to check
`vsp: command not found`	Executable directory is not in PATH	Use the full executable path
`libXYZ.so: cannot open shared object file`	Missing Linux shared library	Run `ldd` and obtain the missing Ubuntu package
`No module named vsp`	Python API path not configured	Check `PYTHONPATH`
`undefined symbol` during import	API and library versions do not match	Rebuild the API against the correct OpenVSP release

`wrong ELF class`	Wrong architecture	Use AMD64 files on AMD64 Ubuntu
`Python version mismatch`	API was compiled for another Python version	Build the API against Python 3.11.9
VSPAERO not found	Incorrect solver path	Check `VSPROOT` and `SetVSPAEROPath`
OpenVSP GUI does not start	Missing GUI library or display dependency	Run `ldd` on the `vsp` executable

28. Final verification checklist

Use this checklist after installation.

- [ ] Ubuntu 24.04 confirmed.
- [ ] Architecture confirmed as `x86_64`.
- [ ] Python 3.11.9 installed.
- [ ] Python 3.11.9 virtual environment created.
- [ ] OpenVSP `.deb` installed.
- [ ] OpenVSP GUI launches.
- [ ] Test aircraft `.vsp3` opens.
- [ ] Python API imports successfully using Python 3.11.9.
- [ ] `vsp.VSPCheckSetup()` succeeds.
- [ ] `vsp.GetVSPVersion()` returns the expected version.
- [ ] VSPAERO executable found.
- [ ] `vsp.SetVSPAEROPath()` succeeds.
- [ ] VSPAERO single-point analysis completes.

29. Important corrections before relying on the offline bundle

There are three things you must verify rather than assume:

1. APT package completeness

The commands in Sections 6–8 collect packages from the connected computer's APT cache. If a dependency is already installed on Computer A, APT may not download another copy of it.

For that reason, after transferring the bundle, test dependency availability on Computer B before installing:

```
sudo apt-get --no-download -s install \
~/OPENVSP_OFFLINE_BUNDLE/01_OPENVSP/OpenVSP-3.56.0-Ubuntu-24.04_amd64.deb
```

This is a simulation, not an installation. If it reports missing packages, return to Computer A and obtain those packages. Do not proceed until the simulation resolves the dependencies.

For the strongest result, prepare the bundle against the same Ubuntu 24.04 update level and architecture as Computer B.

2. Python API compatibility

A successful OpenVSP GUI installation does not prove that the Python API works. The API must import successfully from the Python 3.11.9 virtual environment. If it does not, you need a matching source build.

3. Linux library files

You do not need Windows DLLs. Ubuntu uses `.so` shared libraries. Do not copy Windows `.dll` or `.pyd` files into the Ubuntu installation.

30. Official references

These are the official resources to use when preparing the bundle:

1. OpenVSP official downloads ↗ — Ubuntu 24.04 release package.
2. OpenVSP source code ↗ — source and build information.
3. OpenVSP Ubuntu build instructions ↗ — source-build guidance.
4. OpenVSP Python API documentation ↗ — API setup and functions.
5. Python 3.11.9 official download ↗ — Python source archive.

My recommendation for your installation

Use this order:

1. Download the official OpenVSP Ubuntu 24.04 `.deb` .
2. Download Python 3.11.9 source.
3. Collect all Ubuntu runtime and development dependencies on Computer A.
4. Transfer the bundle to the offline computer.
5. Install Python 3.11.9.
6. Install OpenVSP.
7. Test the GUI.
8. Test the Python API.
9. If the API is incompatible, build the API against Python 3.11.9.
10. Configure and test VSPAERO.

One important limitation: I cannot honestly guarantee that the prebuilt Ubuntu `.deb` contains a Python 3.11.9-compatible API without inspecting that exact downloaded package. The steps above include checks to identify that situation instead of silently installing an incompatible module.

For the next stage, the most useful information is the output of these two commands on Computer A:

```
dpkg-deb -f OpenVSP-3.56.0-Ubuntu-24.04_amd64.deb Depends
```

```
dpkg-deb -c OpenVSP-3.56.0-Ubuntu-24.04_amd64.deb | grep -E 'vsp.py|_vsp|vspaero|/vsp$'
```

Those outputs will establish which dependencies and Python API files are actually in your downloaded package before you transfer it to the offline Ubuntu PC.